Version Control

Version Control

Commits

A Git commit is essentially a snapshot of your project at a specific point in time. When you make changes to your files and want to save those changes in your Git repository, you create a commit. Each commit records the state of your project, along with a unique identifier (a hash), the author's information, and a commit message that describes the changes made.

Here are some tips to improve your commit messages:

- https://cbea.ms/git-commit/
- https://www.conventionalcommits.org/en/v1.0.0/
- https://learntheweb.courses/topics/commit-message-cheat-sheet/

Version Control

Git Workflows

Feature Branch Flow

The Feature Branch Workflow is a branching model in version control systems, such as Git, where developers create a separate branch for each new feature or bug fix. This branch is created off the main branch (often called *main* or *master*) and serves as a dedicated space for development. Once the feature is complete and tested, it can be merged back into the main branch.

Key Steps in the Feature Branch Workflow:

1. **Create a Feature Branch**: When starting work on a new feature, developers create a new branch from the main branch. This branch is typically named descriptively, such as feature/login-system or bugfix/issue-123.
2. **Develop the Feature**: Developers work on the feature within their branch, making commits as they progress. This isolation allows for experimentation and changes without affecting the main codebase.
3. **Regularly Sync with Main**: To avoid conflicts and ensure compatibility, developers should regularly pull changes from the main branch into their feature branch. This practice helps keep the feature branch up to date with the latest code.
4. **Testing**: Once the feature is complete, it should be thoroughly tested. This can include unit tests, integration tests, and user acceptance testing to ensure the feature works as intended.
5. **Merge Back to Main**: After testing, the feature branch can be merged back into the main branch. This is often done through a pull request, allowing for code review and discussion before integration.
6. **Delete the Feature Branch**: Once the feature has been successfully merged, the feature branch can be deleted to keep the repository clean and organized.

To learn more about Feature Branch, check this out

Version Control

Git Flow

Git Flow is a branching strategy that defines a set of rules for managing branches in a Git repository. It emphasizes the use of specific branch types for different purposes, making it easier to track features, releases, and hotfixes.

The main branches in Git Flow are:

- ***main***: This branch contains the production-ready code. It should always reflect the latest stable release.
- ***develop***: This is the integration branch where all feature branches are merged. It serves as a staging area for the next release.
- **Feature Branches**: These branches are created from the *develop* branch to work on new features. Once a feature is complete, it is merged back into *develop*.
- **Release Branches**: When the *develop* branch is ready for a new release, a release branch is created. This branch allows for final adjustments and bug fixes before merging into *main*.
  Hotfix Branches: If a critical bug is found in the *main* branch, a hotfix branch is created to address the issue quickly. Once the fix is implemented, it is merged back into both *main* and *develop*.

To learn more about Git Flow, check this out

Version Control

Forking Flow

Forking Flow is a branching strategy based on the concept of "forking" a repository. When a developer wants to contribute to a project, they create a personal copy (or fork) of the original repository. This allows them to work independently on their changes without affecting the main codebase. Once the changes are complete, the developer can submit a pull request to propose merging their changes back into the original repository.

Key Steps in Forking Flow:

1. **Fork the Repository**: The first step is to create a fork of the original repository on a platform like GitHub. This creates a personal copy of the project under your account.
2. **Clone the Fork**: Then clone the forked repository to your local machine, allowing you to work on the code.
3. **Create a Feature Branch**: Before making changes, it's best practice to create a new branch for the specific feature or bug fix. This keeps the work organized and separate from the *main* branch.
4. **Make Changes and Commit**: After making the necessary changes, commit them to their feature branch, and push the updates to your forked repository.
5. **Submit a Pull Request**: Once the changes are ready, you can submit a pull request to the original repository, proposing that your changes be merged into the *main* codebase.
6. **Review and Merge**: The maintainers of the original repository review the pull request. If everything looks good, they merge the changes, integrating the new code into the project.

To learn more about Forking Flow, check this out